Technical Proposal for QueryMate

An Android-Based LLM Interface for Natural Language Tabular Data Interrogation

Introduction

QueryMate is an Android application engineered to democratize data analysis by enabling users to interrogate tabular datasets via natural language. Its Unique Selling Proposition is a Python-backed Llama ReAct loop that autonomously writes and executes analysis scripts in real-time. Unlike complex BI dashboards dominating the 2026 mobile market, QueryMate delivers instant, conversational insights for raw CSVs, such as microclimate sensor data, spatial metrics, and financial data directly from a smartphone.

Objective

The primary objective is to eliminate the technical barriers of data science for non-programmers. By integrating Llama, the application dynamically translates plain-English queries into executable Python code, effectively solving the inability of mobile users to rapidly extract actionable, on-the-go insights from raw CSVs without heavy dashboarding infrastructure.

Advantage Over Standard LLM Interfaces

Feeding raw tabular data directly into a standard conversational LLM, such as ChatGPT, often results in data hallucinations, as these models are probabilistic text generators rather than deterministic calculators. Furthermore, standard LLMs are constrained by strict token limits, making it impossible to input large CSVs directly into a chat window. QueryMate circumvents these critical flaws by utilizing the LLM solely as a reasoning engine to write Python code. The actual computation is performed by the embedded pandas library, guaranteeing 100% mathematically accurate results and bypassing the token limitations of traditional chat interfaces.

Proposed Solution

QueryMate delivers a fully localized analytical pipeline, shifting computational power entirely to the edge to eliminate cloud dependency.

Core Features & Interactions

The user journey begins on the Data Ingestion screen, where users upload raw tabular files, such as Melbourne microclimate sensor datasets. The application then transitions to the Interrogation Interface, a chat-based UI. When a user inputs a query like, "What is the peak temperature variance?", the system processes the request locally.

On-Device LLM Integration

Utilizing a quantized Llama model deployed natively on Android, the LLM functions as a localized reasoning agent. It evaluates the CSV schema and translates the natural language query into executable Python pandas scripts. Leveraging Chaquopy, the app bridges its Java frontend with an embedded Python environment, executing the generated scripts on-device to extract precise quantitative insights without hallucinating data values.

Benefits of Llama

Deploying Llama at the edge guarantees zero latency and full offline capability, a critical advantage for field researchers operating in environments without reliable network access. Furthermore, it democratizes decision support while ensuring absolute data privacy, as proprietary or sensitive datasets never leave the user's smartphone.

Safety & Quality Controls

To maintain deterministic outputs, strict system-level prompt rules constrain the LLM exclusively to data-centric code generation. Automated refusal cases are triggered if a user attempts general Q&A or requests non-analytical content. Additionally, the embedded Python environment is strictly sandboxed, neutralizing system-level permissions to prevent malicious code execution and ensure secure, moderated data handling.

Technology Stack & Deployment Strategy

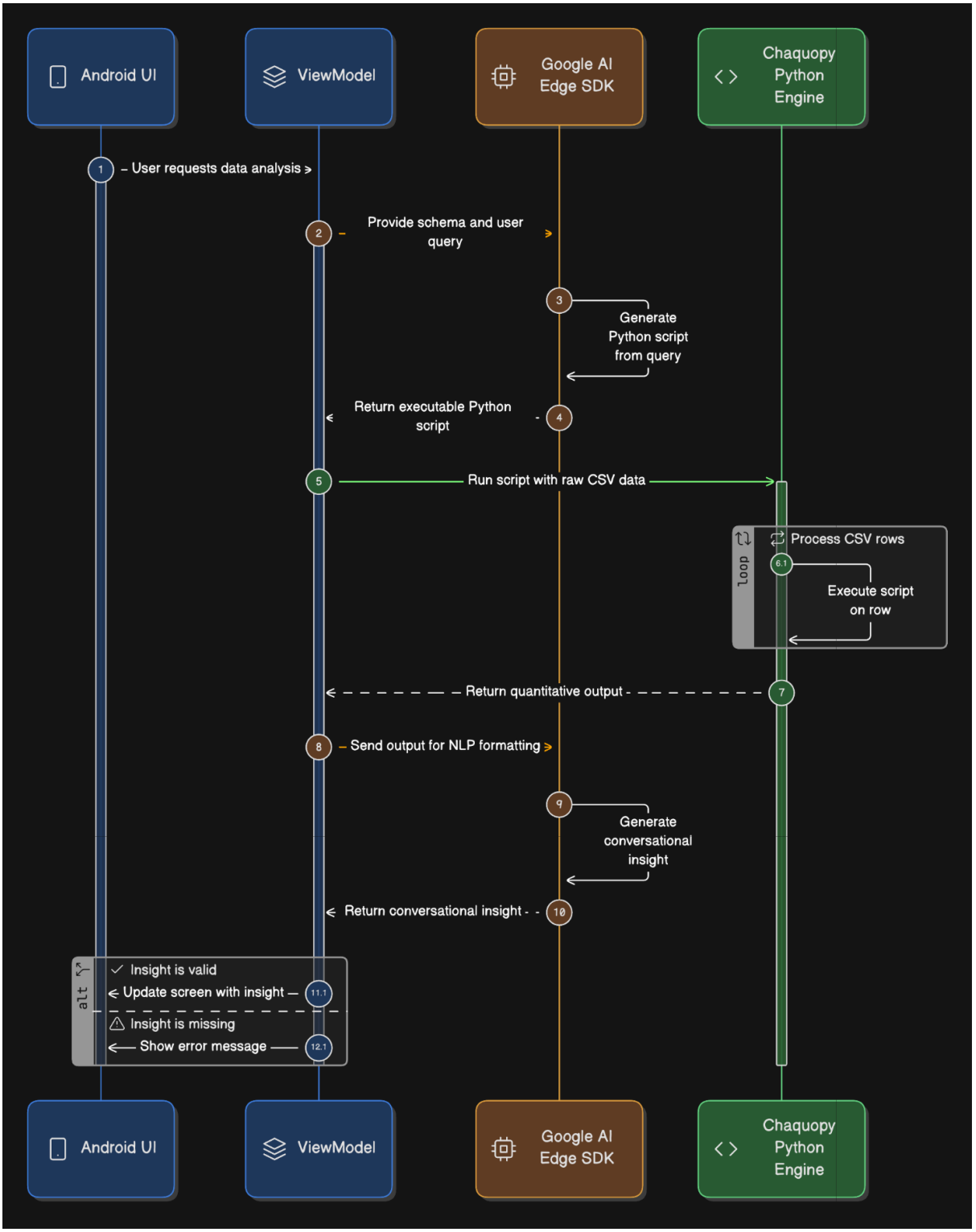
- Development Environment - Android Studio.
- Frontend Application - Kotlin/Java utilizing the MVVM (Model-View-ViewModel) architectural pattern to ensure a clean separation of UI and business logic.

- AI Inference Engine - Google AI Edge SDK. This will run a quantized (compressed) Llama model directly on the device's NPU/GPU.
- Execution Environment - Chaquopy. This plugin embeds a lightweight Python environment (including the pandas library) within the Android app, enabling localized script execution.

Deployment Mode - Strictly On-Device (Edge Computing)

The application is deployed with a completely localized infrastructure. The quantized Llama model is either bundled with the APK or downloaded upon initial setup. This strictly on-device deployment eliminates cloud server costs, guarantees zero network latency, and ensures absolute data privacy, as sensitive datasets never leave the user's mobile device.

System Architecture & Interaction Diagram



Feasibility - Assumptions & Prerequisites

To ensure this on-device architecture is highly feasible and performs reliably, the following assumptions are made:

- Hardware Prerequisites - Target devices must have a modern NPU/GPU and at least 6–8GB of RAM to efficiently run the quantized Llama model without performance or thermal issues.
- Dataset Dimensions - Uploaded tabular datasets are assumed to be under 20MB to ensure they can be processed fully in-memory without causing application crashes.
- Data Cleanliness - The system assumes imported CSV files are well-formatted with clear headers and standard delimiters, reducing the need for complex preprocessing before analysis.

Development Methodology

An Agile/Scrum methodology will be adopted. This iterative approach is essential for experimental AI integration, where prompt engineering and on-device Llama execution require continuous refinement. Agile facilitates frequent testing of the self-correcting ReAct loop, early risk mitigation regarding hardware memory constraints, and accommodates regular tutor feedback to ensure the project meets academic milestones phase by phase.

Conclusion

QueryMate redefines mobile data analysis by deploying zero-latency, privacy-first LLM reasoning directly to the edge. By executing Python scripts locally on Android, it transforms smartphones into autonomous, offline analytical engines, eliminating cloud dependency and bridging the gap between raw datasets and instant insights.